JAVA SCRIPT
CHAPTER-4

4.1 JAVASCRIPT ARRAY METHODS

Array methods are functions built-in to JavaScript that we can apply to our arrays — each method has a unique function that performs a change or calculation to our array and saves us from writing common functions from scratch.

Array To string ()
The JavaScript method to String () converts an array to a string of (comma separated) array values.

PROGRAM 1:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Arrays</h1>
<h2>The toString() Method</h2>
<p>The toString() method returns an array as a comma separated string:</p>
<p id="demo"></p>
<script>
const fruits = ["Banana", "Orange", "Apple", "Mango"];
document.getElementById("demo").innerHTML = fruits.toString();
</script>
</body>
</html>
```

Popping and Pushing

The pop () method removes the last element from an array. Push () method adds a new element to an array (at the end).

PROGRAM 2:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Arrays</h1>
<h2>The push() Method</h2>
<p>The push() method appends a new element to an array:</p>
<p id="demo1"></p>
<p id="demo2"></p>
<script>
const fruits = ["Banana", "Orange", "Apple", "Mango"];
document.getElementById("demo1").innerHTML = fruits;
```

```
fruits.push("Kiwi");
//fruits.pop
document.getElementById("demo2").innerHTML = fruits;
</script>
</body>
</html>
```

Array shift () and Array unshift ()

The shift () method removes the first array element and "shifts" all other elements to a lower index. Unshift () method adds a new element to an array (at the beginning), and "unshifts" older elements.

PROGRAM 3:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Arrays</h1>
<h2>The unshift() Method</h2>
<p>The unshift() method adds new elements to the beginning of an array:</p>
<p id="demo1"></p>
<p id="demo2"></p>
<script>
const fruits = ["Banana", "Orange", "Apple", "Mango"];
document.getElementById("demo1").innerHTML = fruits;
fruits.unshift("Lemon");
//fruits.shift("Lemon");
document.getElementById("demo2").innerHTML = fruits;
</script>
</body>
</html>
```

Merging (Concatenating) Arrays

The concat () method creates a new array by merging (concatenating) existing arrays:

PROGRAM 4:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Arrays</h1>
<h2>The concat() Method</h2>
<p>The concat() method merges (concatenates) arrays:</p>
<p id="demo"></p>
<script>
const myGirls = ["Cecilie", "Lone"];
```

```
const myBoys = ["Emil", "Tobias", "Linus"];
const myChildren = myGirls.concat(myBoys);
document.getElementById("demo").innerHTML = myChildren;
</script>
</body>
</html>
```

Array splice ()

The splice () method can be used to add new items to an array:

PROGRAM 5:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Arrays</h1>
<h2>The splice() Method</h2>
<p>The splice() method adds new elements to an array:</p>
<p id="demo1"></p>
<p id="demo2"></p>
<script>
const fruits = ["Banana", "Orange", "Apple", "Mango"];
document.getElementById("demo1").innerHTML = fruits;
fruits.splice(2, 0, "Lemon", "Kiwi");
document.getElementById("demo2").innerHTML = fruits;
</script>
</body>
</html>
```

Array slice ()

The slice () method slices out a piece of an array into a new array. This example slices out a part of an array starting from array element 1 ("Orange"):

PROGRAM 6:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Arrays</h1>
<h2>The slice() Method</h2>
<p>Slice out a part of an array starting from array element 1 ("Orange"):</p>
<p id="demo"></p>
<script>
const fruits = ["Banana", "Orange", "Lemon", "Apple", "Mango"];
const citrus = fruits.slice(1);
document.getElementById("demo").innerHTML = fruits + "<br><br>" + citrus;
</script>
</body>
```

```
</html>
```

4.2 SORTING AN ARRAY

The sort () method sorts an array alphabetically:

PROGRAM 7:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Arrays</h1>
<h2>The sort() Method</h2>
<p>The sort() method sorts an array alphabetically:</p>
<p id="demo1"></p>
<p id="demo2"></p>
<script>
const fruits = ["Banana", "Orange", "Apple", "Mango"];
document.getElementById("demo1").innerHTML = fruits;
fruits.sort();
document.getElementById("demo2").innerHTML = fruits;
</script>
</body>
</html>
```

Numeric Sort

By default, the sort () function sorts values as strings. This works well for strings ("Apple" comes before "Banana").However, if numbers are sorted as strings, "25" is bigger than "100", because "2" is bigger than "1".Because of this, the sort() method will produce incorrect result when sorting numbers.

PROGRAM 8:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Arrays</h1>
<h2>The sort() Method</h2>
<p>Sort the array in ascending order:</p>
<p id="demo1"></p>
<p id="demo2"></p>
<script>
const points = [40, 100, 1, 5, 25, 10];
document.getElementById("demo1").innerHTML = points;
points.sort(function(a, b){return a - b});
document.getElementById("demo2").innerHTML = points;
</script>
</body>
</html>
```

The Compare Function

The purpose of the compare function is to define an alternative sort order. The compare function should return a negative, zero, or positive value, depending on the arguments:

```
Function (a, b) {return a - b}
```

When the sort () function compares two values, it sends the values to the compare function, and sorts the values according to the returned (negative, zero, positive) value.

If the result is negative, a is sorted before b.

If the result is positive, b is sorted before a.

If the result is 0, no changes are done with the sort order of the two values.

PROGRAM 9:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Arrays</h1>
<h2>The sort() Method</h2>
<p>Click the buttons to sort the array alphabetically or numerically.</p>
<button onclick="myFunction1()">Sort Alphabetically</button>
<button onclick="myFunction2()">Sort Numerically</button>
<p id="demo"></p>
<script>
const points = [40, 100, 1, 5, 25, 10];
document.getElementById("demo").innerHTML = points;
function myFunction1() {
  points.sort();
  document.getElementById("demo").innerHTML = points;
}
function myFunction2() {
  points.sort(function(a, b){return a - b});
  document.getElementById("demo").innerHTML = points;
}
</script>
</body>
</html>
```

Sorting an Array in Random Order

PROGRAM 10:

```
<!DOCTYPE html>
<html>
```

```
<body>
<h1>JavaScript Arrays</h1>
<h2>The sort() Method</h2>
<p>Click the button (again and again) to sort the array in random order.</p>
<button onclick="myFunction()">Try it</button>
<p id="demo"></p>
<script>
const points = [40, 100, 1, 5, 25, 10];
document.getElementById("demo").innerHTML = points;
function myFunction() {
  points.sort(function(){return 0.5 - Math.random()});
  document.getElementById("demo").innerHTML = points;
}
</script>
</body>
</html>
```

Using Math.max () on an Array

You can use Math.max. Apply to find the highest number in an array:

PROGRAM 11:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Array Sort</h1>
<p>The highest number is <span id="demo"></span>.</p>
<script>
const points = [40, 600, 1, 5, 25, 10];
document.getElementById("demo").innerHTML = myArrayMax(points);
function myArrayMax(arr) {
  return Math.max.apply(null, arr);
}
</script>
</body>
</html>
```

Using Math.min () on an Array

You can use Math.min. Apply to find the lowest number in an array:

PROGRAM 12:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Array Sort</h1>
<p>The lowest number is <span id="demo"></span>.</p>
<script>
const points = [40, 100, 1, 5, 25, 10];
document.getElementById("demo").innerHTML = myArrayMin(points);
```

```
function myArrayMin(arr) {
  return Math.min.apply(null, arr);
}
</script>
</body>
</html>
```

Sorting Object Arrays

JavaScript arrays often contain objects: Even if objects have properties of different data types, the sort () method can be used to sort the array. The solution is to write a compare function to compare the property values:

PROGRAM 13:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Arrays</h1>
<h2>The sort() Method</h2>
<p>Sort car objects on age:</p>
<p id="demo"></p>
<script>
const cars = [
  {type:"Volvo", year:2016},
  {type:"Saab", year:2001},
  {type:"BMW", year:2010}
];
displayCars();
cars.sort(function(a, b){return a.year - b.year});
displayCars();
function displayCars() {
  document.getElementById("demo").innerHTML =
  cars[0].type + " " + cars[0].year + "<br>" +
  cars[1].type + " " + cars[1].year + "<br>" +
  cars[2].type + " " + cars[2].year;
}
</script>
</body>
</html>
```

4.3 Creating Date Objects

Date objects are created with the new Date () constructor. There are **9 ways** to create a new date object:

```
new Date()
new Date(date string)
```

```
new Date(year,month)
new Date(year,month,day)
new Date(year,month,day,hours)
new Date(year,month,day,hours,minutes)
new Date(year,month,day,hours,minutes,seconds)
new Date(year,month,day,hours,minutes,seconds,ms)
new Date(milliseconds)
```

PROGRAM 14:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Dates</h1>
<h2>Using new Date()</h2>
<p>Create a new date object with the current date and time:</p>
<p id="demo"></p>
<script>
const a = new Date();                    (new date)
const b = new Date("October 13, 2014 11:13:00");    (date string)
const c= new Date(2018, 11, 24, 10, 33, 30, 0);        (year,month)
const d = new Date(2018, 11, 24);                       (year,month,day)
const e = new Date(2018, 11, 24, 10);                  (year,month,day,hours)
const f = new Date(2018, 11, 24, 10, 33);               (year,month,day,hours,minutes)
const g = new Date(2018, 11, 24, 10, 33, 30);
(year,month,day,hours,minutes,seconds)
document.getElementById("demo").innerHTML = d;
</script>
</body>
</html>
```

New Date (*Milliseconds*)

New Date (*milliseconds*) creates a new date object as **milliseconds** plus zero time:

PROGRAM 15:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Dates</h1>
<h2>Using new Date()</h2>
<p>100000000000 milliseconds from January 01 1970 UTC is:</p>
<p id="demo"></p>
<script>
const d = new Date(100000000000);
```

```
document.getElementById("demo").innerHTML = d;
</script>
</body>
</html>
```

4.4 DATE FORMATS

Date Input

There are generally 3 types of JavaScript date input formats:

Type	Example
ISO Date	"2015-03-25" (The International Standard)
Short Date	"03/25/2015"
Long Date	"Mar 25 2015" or "25 Mar 2015"

PROGRAM 16:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript ISO Dates</h2>
<p id="demo"></p>
<script>
const a = new Date("2015-03-25");
const b = new Date("2015-03");   //year and month
const c= new Date("2015"); // only year
const d = new Date("2015-03-25T12:00:00Z");//date-time
const e = new Date("03/25/2015");//short date
const g = new Date("Mar 25 2015");//long date
document.getElementById("demo").innerHTML = d;
</script>
</body>
</html>
```

4.4 Get Date Methods

Method	Description
getFullYear()	Get **year** as a four digit number (yyyy)
getMonth()	Get **month** as a number (0-11)
getDate()	Get **day** as a number (1-31)
getDay()	Get **weekday** as a number (0-6)
getHours()	Get **hour** (0-23)
getMinutes()	Get **minute** (0-59)
getSeconds()	Get **second** (0-59)
getMilliseconds()	Get **millisecond** (0-999)
getTime()	Get **time** (milliseconds since January 1, 1970)

PROGRAM 17:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Dates</h1>
<p>Return the full year of a date object:</p>
<p id="demo"></p>
<script>
const a = new Date("2021-03-25")
document.getElementById("demo").innerHTML = d.getFullYear();
//d.getMonth() + 1;
/ d.getDate();
//d.getHours();
//d.getMinutes();
//d.getSeconds();
//d.getMilliseconds();
//d.getDay();
//d.getTime();
</script>
</body>
</html>
```

The Date. Now () Method

Date. Now () returns the number of milliseconds since January 1, 1970.

PROGRAM 18:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Dates</h1>
```

```
<h2>The Date.now() Method</h2>
<p>Return the current date/time in milliseconds since January 1, 1970:</p>
<p id="demo"></p>
<script>
const date = Date.now();
document.getElementById("demo").innerHTML = date;
</script>
</body>
</html>
```

4.5 SET DATE METHODS

Method	Description
setDate()	Set the day as a number (1-31)
setFullYear()	Set the year (optionally month and day)
setHours()	Set the hour (0-23)
setMilliseconds()	Set the milliseconds (0-999)
setMinutes()	Set the minutes (0-59)
setMonth()	Set the month (0-11)
setSeconds()	Set the seconds (0-59)
setTime()	Set the time (milliseconds since January 1, 1970)

4.6 MATH METHODS

The syntax for Math any methods is: Math. *Method* (*number*)

Number to Integer

There are 4 common methods to round a number to an integer:

```
Math.round(x)Returns x rounded to its nearest integer
Math.ceil(x)    Returns x rounded up to its nearest integer
Math.floor(x)  Returns x rounded down to its nearest integer
Math.trunc(x)  Returns the integer part of x (new in ES6)
```

PROGRAM 19:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript math methods</h2>
<p>Math.round(x) returns the value of x rounded to its nearest integer:</p>
<p id="demo"></p>
<script>
document.getElementById("demo").innerHTML = Math.round(9.8);
// Math.ceil(x)
// Math.floor(x)
// Math.trunc(4.7)
```

```
</script>
</body>
</html>
```

Math. POW ()

Math. POW(x, y) returns the value of x to the power of y:

PROGRAM 20:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Math.pow()</h2>
<p>Math. Pow(x,y) returns the value of x to the power of y:</p>
<p id="demo"></p>
<script>
document.getElementById("demo").innerHTML = Math.pow(8,2);
// Math.sqrt(64)
// Math.abs(-4.7)
</script>
</body>
</html>
```

Math. Sin () and Math.cos ()

Math. Sin(x) and Math.cos(x) returns the sine (a value between -1 and 1) of the angle x (given in radians).If you want to use degrees instead of radians, you have to convert degrees to radians:

Angle in radians = Angle in degrees x PI / 180.

PROGRAM 21:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Math.sin()</h2>
<p>Math.sin(x) returns the sin of x (given in radians):</p>
<p>Angle in radians = (angle in degrees) * PI / 180.</p>
<p id="demo"></p>
<script>
document.getElementById("demo").innerHTML =
"The sine value of 90 degrees is " + Math.sin(90 * Math.PI / 180);
// Math.cos(0 * Math.PI / 180)
</script>
</body>
</html>
```